ChatGPT

Monitor Legislativo v4 – System Diagnostic Audit

Application Structure & Version Handling (app.R)

The Shiny app's **structure is robust**, loading features modularly and patching known bugs in R's version handling. For example, `app.R` overrides the base `compareVersion` to safely handle `NULL/NA` inputs [1]. This prevents crashes from version checks by treating missing versions as `"0.0.0"` – a proactive fix for earlier compareVersion bugs. The app also binds to the correct host/port in production mode [2], ensuring it listens on Railway's interface.

Module Loading: All major feature modules (Executive Summary, Library, Analytics, Geographic, Maps, São Paulo, Text) are loaded conditionally with error capture [3] [4]. This design means a failure in any module's code won't stop the whole app – the module is simply marked as unavailable and a fallback UI is used. The UI menu even indicates which modules loaded ("✓") vs fell back ("!") [5] [6]. Each tab checks if its module's UI function exists and otherwise shows an alternate interface [7] [8]. This yields a graceful degradation: for example, if the Geographic module fails, the *Geographic Analysis* tab displays a descriptive placeholder and a warning instead of breaking the app [9].

⚠ **DB Status Indicator:** One minor logic issue is the system status always showing "Database: Connected" as long as the `get_documents` function exists [10]. Since `get_documents` is defined unconditionally in global scope, the app may report **Connected** even if it's actually using a CSV fallback. **Root Cause:** The status check uses the existence of the function rather than an actual connection test [11]. **Fix:** Adjust the condition to reflect real DB connectivity – for example, set a flag when the pool connects successfully, or attempt a lightweight query (as done in the pool startup) and update the status accordingly.

⚠ **Fallback UI Edge Cases:** The fallback UIs for modules are generally effective, but a few edge cases exist. For instance, the *Interactive Maps* fallback simply shows a basic leaflet map of Brazil [12] [13]. This works, but without the richer polygon data intended for that tab. Similarly, the *Advanced Analytics* fallback is a static description with a warning if the module is missing [14] [15]. These are not bugs per se, but they indicate those modules did not load. We address their causes below.

Global Initialization & Database Pool (global.R)

The **global configuration** script sets up critical environment-dependent features and the database pool. It loads essential libraries and configures monitoring, caching, and security settings. Environment usage is thorough: e.g. `app_config$environment` comes from `RAILWAY_ENVIRONMENT` (defaulting to "development") [16], and feature flags like `use_demo_data` and `monitoring_enabled` are read from env vars [16]. The **database config** is assembled from environment variables as well – it prefers a unified `DATABASE_URL` if provided, or falls back to individual `PGHOST`, `PGUSER`, etc. [17] [18]. This means as long as the Railway service or .env sets either the full connection string or the standard PG vars, the app will pick them up correctly.

Environment & Pool Setup: The app creates a persistent PostgreSQL connection pool at startup. After sourcing the `data_service` module, it calls `get_db_connection()` to initialize the pool [19] [20]. The pool uses `RPostgres::Postgres()` and is configured with sane defaults (min 2, max 8 connections, 10-minute idle timeout) [21] [22]. Importantly, it runs a startup query (`SELECT 1`) and even attempts to count records in the main table to verify the connection [23] [24]. This ensures that if the DB is unreachable or the table is missing, it fails fast (logging an error) rather than hanging. Any failure in pool creation is caught and results in no pool (NULL) [25] – the app will later fall back to CSV seamlessly in that case. The code also sets session parameters on the DB connection (lock timeout, statement timeout, etc.) for reliability [26].

Monitoring & Logging: A custom monitoring subsystem is integrated. If present, `monitoring/logger.R` and related files are sourced and initialized [27] [28]. The logger is set to be Railway-compatible (sanitizing sensitive info) and a telemetry system is started (though `start_monitoring()` is currently disabled) [29]. This means the app can log events (e.g. errors in data metrics) to console or future endpoints. Notably, the code has a switch `ENABLE_QUERY_MONITORING` flag [30], though we did not find active usage of this flag in the query logic – it appears to be a placeholder for potential query performance tracking.

⚠ **Missing Package Handling:** In `global.R`, all **essential packages** are loaded at the top without protection [31]. If any of these (e.g. `stringr`, `RPostgres`, `shinydashboard`) failed to install in the image, the app would error out on startup. For instance, an absent **stringr** package would cause `library(stringr)` to throw an error, preventing the app from launching. **Root Cause:** These libs are assumed to be installed in production, but if the Docker build missed one, there's no fallback. (In contrast, a set of optional packages like `scales`, `sf`, `geobr`, `leaflet` are loaded inside a try-catch, printing a warning and continuing if they're missing [32].) Indeed, the logs likely show messages such as *"Package sf not available, continuing without it"* [33], indicating those weren't present at runtime. **Fix:** Ensure all essential packages are included in the install script (see **Packages** section). Additionally, consider wrapping non-critical libraries in try-catch so the app can start even if one is missing – similar to how optional packages are handled.

⚠ **Monitoring Startup Bug:** The monitoring system's `start_monitoring()` is commented out due to 10-minute hang issues [34]. **Cause:** This function likely spawns a background observer (perhaps for performance metrics) that was causing the app to stall (possibly waiting on a blocking call or mis-configured interval). As a result, `monitoring_enabled` is set to TRUE only after partial init [28] [35], and logging of app start occurs, but live monitoring of metrics is not running. **Fix:** Further investigate the `start_monitoring` implementation – it may need to run in a non-blocking future or be adjusted for Shiny's reactive context. For now, the decision to disable it is wise (to keep the app responsive), but this should be revisited to fully utilize the monitoring features.

⚠ **Query Monitoring Flag:** As noted, the `ENABLE_QUERY_MONITORING` environment flag is defined [30] but not actually used in the data queries. There's no code toggling detailed query logging or performance metrics based on this flag in the current version. This is a minor issue (no impact on functionality), but it means a feature advertised by the config isn't active. **Fix:** Implement query monitoring if needed – e.g., wrap `dbGetQuery` calls in timing logic to log slow queries when `ENABLE_QUERY_MONITORING=true`, or remove the unused flag to avoid confusion.

Database Layer & Data Service (modules/data_service.R)

All database operations are funneled through a unified **data service module**, which greatly simplifies error handling and fallback logic. **Confirmed workings:**

- **Connection Handling:** The `data_service` uses a single persistent pool (exported as `db_connection_pool`) and exposes a `get_db_connection()` helper [36]. On first use, it creates the pool via `create_db_pool()` [20]. Subsequent calls reuse the existing pool (no constant reconnect churn). The pool creation logic is solid – it tries using `DATABASE_URL` if available, otherwise builds a connection from `PGHOST/PGUSER/...` [17] [37]. If a `DATABASE_URL` was set in Railway, the code logs that and uses it [38], which is convenient. The pool is tested immediately with a simple query [39] to ensure connectivity.

- **Automatic Fallback to CSV:** One of the strongest aspects is the transparent fallback to local data if the database is unavailable or empty. The main `get_documents()` function first attempts a DB query and catches any error [40] [41]. If the DB query fails **or returns 0 rows**, it logs a warning and then tries `get_documents_from_csv` [42]. That function will load a local CSV file (`data/lexml_unified_dataset.csv`) and apply the same filtering logic [43] [44]. If the CSV is present and has data, the app uses it and logs that it "Retrieved XYZ documents from CSV fallback" [45]. This means the live dashboard will still show data (perhaps slightly stale) even if the database connection fails – exactly as the user expects for a resilient system. If the CSV is also missing or empty, `get_documents()` ultimately returns an empty dataframe with the correct columns [46] [47] so that the UI can handle it gracefully. In short, **the app will serve data from the DB when possible and seamlessly switch to a static dataset when not**, which is a major reliability win.

- **Demo Mode:** There's also an explicit demo mode triggered by `USE_DEMO_DATA=true` (intended for testing or if no real data is available) [48] [49]. In demo mode, `get_documents()` bypasses the database entirely and loads a small demo CSV or synthesizes a dataset on the fly [49] [50]. This is confirmed by log messages (e.g. *"Using demo data (USE_DEMO_DATA=true)"* in the code [51]). This feature appears to work as designed and is off by default in production.

⚠ **Package/Driver Issues:** To fully leverage the DB layer, certain environment details must be correct. Notably, if using an external Postgres (like Supabase) the SSL mode needs attention. In `global.R` the default `PGSSLMODE` is set to "require" [52], but in `data_service$get_db_config()` it defaults to "disable" if not specified [18]. If `DATABASE_URL` is used, the sslmode might be embedded in that URL, but if not, there's a risk of connection failure (Supabase requires SSL). **Root Cause:** A slight inconsistency in default SSL settings between global config and data_service. **Fix:** Ensure that for external DBs, `PGSSLMODE=require` is set in the environment (or append `?sslmode=require` in the `DATABASE_URL`). In a Railway internal Postgres scenario, `disable` is fine – but the code comment suggests it's assuming an internal DB [53]. It's safest to explicitly match the environment: if using Supabase, set `PGSSLMODE=require` so the pool connects properly in R (or rely purely on `DATABASE_URL` which the code will prefer).

⚠ **Fallback Data Freshness:** When operating in fallback mode, the app relies on the static CSV. If the production DB updates, the CSV can become outdated. There is currently **no cache invalidation or update mechanism** for that CSV once the app is running. Also, each query will re-read the CSV from disk. For large

datasets, repeatedly loading the CSV can be slow. **Root Cause:** The design opts for simplicity (always reading file) over in-memory caching. **Fix:** To improve performance, consider loading the CSV once (e.g., on startup if DB is down, or cache the first `get_documents_from_csv` result in a global variable). Alternatively, use `data.table::fread` (since `data.table` is listed as optional) for faster reads and filtering, or implement a simple in-memory cache inside `get_documents` (e.g., don't re-read if the previous CSV read was recent and filters are similar). This would make the fallback more efficient during extended DB outages.

⚠ **Query Monitoring & Logging:** The data service does log informational messages for major events (retrieving from DB/CSV, errors, etc.) [54] [55], which is good. However, it doesn't time the queries or log detailed performance metrics. Given the presence of `ENABLE_QUERY_MONITORING` flag, this might be a planned enhancement. **Fix:** If needed, wrap `dbGetQuery` calls with timing (using `Sys.time()` before/after) and log durations when the flag is true. Also, consider logging the number of results fetched, which is partially done (it logs count for DB and CSV fetches already [56]). These tweaks can help identify slow queries or large data transfers in production.

⚠ **Minor Redundancy:** There is a minor redundancy where `global.R` sources `modules/data_service.R` twice (once normally, once via `local(...$value)`) [57]. This doesn't break anything (the second source likely re-defines the same functions in a local env and returns the list of functions), but it's unnecessary. **Fix:** Remove the initial `source("modules/data_service.R")` call – using the second call that assigns `data_service` is enough. This will avoid any potential double-execution of initialization code inside that module.

Feature Modules & Functionalities

Each functional area is implemented as a module (with separate UI and server scripts), and the app either uses the module or falls back. We review the critical ones:

- **Executive Summary:** If available, it provides an overview dashboard. If its module fails to load, the app falls back to showing three key value boxes (Total Docs, States Covered, Municipalities) and a basic plotly charts section [58] [59]. **Working:** The fallback uses `get_lexml_dashboard_metrics()` to populate the KPIs [60], which in turn computes real metrics if data is available [61]. This means even without the dedicated module, the main stats are displayed. We can confirm these metrics update appropriately (they call `nrow(get_documents())` etc. to reflect current data) [62]. There is no known bug here – the design is intentional to ensure core stats always show.

- **Library & Search:** The Library module (enhanced document library) is conditionally loaded. In its absence, the fallback provides a basic search interface and results table [63] [64]. This fallback is functional: it uses a text input and search button, and the server defines a `search_results` reactive that calls `get_documents` with the search term filter [65] [66]. If no search is entered, it displays a prompt in the table [67]. **Confirmed:** This global search works in both DB and CSV modes (since `get_documents` abstracts that). One integration point is the **Advanced Search** filters. If the `advanced_search` module loads, its UI (`advanced_search_filters_ui`) is inserted into the search box area [68], allowing additional filter controls. However, we found that while the UI can appear, the corresponding server logic may not be fully wired. The `advanced_search_server.R`

is sourced on startup [69] but unlike other modules, there is no call like `advanced_search_server(input, output, session)` in `app.R`. **Root Cause:** The advanced search server code might not be executed at runtime, meaning any reactive logic it contains won't run. **Fix:** Invoke the advanced search module's server function after loading (similar to how library or analytics modules are invoked [70]). Alternatively, if the advanced filters simply augment the global `search_results()` logic, ensure that those filter inputs are respected. Currently, the `filters` list in `get_documents()` can take additional parameters (like category, etc.), but the `search_results` reactive only uses the text filter [65]. Integrating advanced filters likely requires modifying `search_results` to include those inputs (e.g., selected category, date range) when present. This is an improvement area to get the full benefit of the advanced search UI.

- **Advanced Analytics (Charts):** The Advanced Analytics module (charts and statistical analysis) is loaded as `analytics_ui` / `analytics_server`. If it fails or is absent, the Analytics tab still shows a description of features and a warning banner [71] [15]. **Issue:** Currently, if the module didn't load, users just see the static text. Likely causes for module failure could be missing packages like `echarts4r` (since the timeline mentions 36+ chart types). We see `echarts4r` in the optional package list [32]; if it wasn't installed, the module code using it would error and trigger fallback. **Fix:** Ensure **echarts4r** is installed (so the module can load). Also verify the module's server code – it might rely on data or functions defined globally. Given that `get_analytics_data()` is provided by the data service [72], the analytics module should use that for inputs. The good news: `get_analytics_data()` returns a preprocessed dataset for all documents (adding year/month, etc.) and is always available [72] [73]. So the analytics module just needs to be present with its UI outputs to display charts. In summary, **installing the missing packages (echarts4r, etc.)** and double-checking any UI output IDs would likely get this module working. No fundamental code bug is evident beyond dependency installation.

- **Geographic Analysis:** This module likely displays maps or geographic breakdowns (the timeline mentioned integration of IBGE data for 5,570 municipalities). The code tries to load either `visualization_ui.R` and a corresponding server (`geographic_server.R` or `spatial_analysis_server.R`) [74] [75]. If loaded, it presumably uses shape data (perhaps via the `geobr` package or shapefiles) to render choropleth maps. If it fails, the tab shows a list of expected features and a warning that the module isn't available [76] [77]. The presence of `sf` and `geobr` in the optional packages is key [32] – if those weren't installed, the geographic module would error out (e.g., trying to `library(sf)` or use `geobr::read_municipality` would fail). **Root Cause:** Missing geospatial libraries or possibly missing shapefile data. **Fix:** Install the **sf** package (which requires the OS libraries already included in the Dockerfile [78]) and the **geobr** package for Brazilian geo data. With those in place, reload the app – it should be able to draw maps. The fallback already includes a basic leaflet map centered on Brasília as a placeholder [13], but the full module would show detailed maps per state/municipality. Also, ensure the `modules/geographic/` files are present and correct – if any path or name mismatch (e.g., the UI function is named differently than expected), adjust the app's conditional. We saw the app checks for `geographic_analysis_ui` or `visualization_ui` [79] [80]; it's worth confirming the module defines one of those. If not, updating the function name or the existence check will resolve the integration.

- **Interactive Maps:** This might overlap with Geographic, but seems intended as a separate "Maps" feature (perhaps an interactive Leaflet where users can explore data). The module is `map_ui.R/map_server.R`. If it loads, the Maps tab should present a rich map (potentially with layers or

markers for legislative data). If not, the fallback currently shows a static Leaflet with a marker on Brasília [12] [13]. The fallback confirms Leaflet itself is installed (it checks `requireNamespace("leaflet")` before rendering [13]). If Leaflet were missing, it would show nothing. Since `leaflet` is in optional packages and likely installed (rocker/shiny includes it by default), that's fine. The more critical dependency here is again **sf** if shapes are used. The map module might try to load a GeoJSON or shapefile of Brazil states via sf. If sf was missing, the module would have failed. **Fix:** After installing sf/geobr as above, the interactive map module should function. Also, consider pre-loading large spatial data once (perhaps in global.R or on module load) rather than each user session, but that's an optimization.

- **São Paulo Analysis:** This is a specialized module for São Paulo. If not loaded, its tab shows a description and a warning [81] [82]. This suggests the module might not be fully implemented or was turned off intentionally. It likely requires a subset of data or analysis specific to São Paulo state, possibly additional data tables. We did not find explicit errors here; it's possibly just incomplete. **Recommendation:** Ensure any data or logic specific to São Paulo (for example, if it needs a separate dataset or shapefile for SP metropolitan region) is provided. If the module code is present but failing, check for missing packages (maybe none specific beyond those mentioned) or functions. Otherwise, this can be left in fallback mode until development is finished.

- **Text Analytics:** The Text Analytics/NLP module would handle things like topic modeling, sentiment, etc. The app tries several possible file names (text_analytics vs legislative_analytics) to load it [83] [84]. If none load, the tab shows a list of expected NLP features and a warning [85] [86]. The fact it's looking for multiple filenames suggests this module was evolving. It may depend on external NLP libraries (e.g., **tidytext**, or even Python via reticulate if doing embeddings). Since we don't see those in the package list, it's possible this module isn't fully operational yet. If this feature is priority, identify what it needs (e.g., if using `udpipe` or `spacyr` for entity recognition, install those). Otherwise, treat it as a future enhancement. There's no harm in it showing the info-only placeholder for now.

Overall, **the module system is well-architected** – no single module's failure brings down the app, and each critical feature has at least a minimal fallback. The primary action items are installing missing packages and hooking up the advanced search logic on the server side. Once those are addressed, all tabs should become fully functional (the vs ⚠ badges in the sidebar will turn green as modules load successfully). The live deployment logs should also then show " [Module] module loaded" for each, instead of any "⚠ [Module] module failed" warnings [87] [88].

Package Dependencies & Deployment Environment

Given the analysis above, the **package installation** is the main factor in unresolved issues:

📦 **Missing R Packages:** Ensure the Docker build (or `.Rprofile` installation script) includes the following packages, which were either missing or marked optional but needed in production:

- `sf` – for spatial data support (required by geographic modules). The Dockerfile already installs system libraries for sf (GDAL, GEOS, PROJ, etc. [78]), but if `install.packages("sf")` wasn't run, it needs to be added.

- `geobr` – for Brazilian geographic data (shapefiles for states/municipalities). This is optional but highly recommended to realize the full mapping features.
- `stringr` – must be installed, as it's used extensively for text filtering and was causing startup errors if absent [31]. It may come with the **tidyverse**, but if only parts of tidyverse were installed, stringr might have been missed. Explicitly include it.
- `echarts4r` – for advanced interactive charts. Without it, the analytics module cannot render those 36+ chart types mentioned.
- `lubridate`, `tidyr`, `data.table` – these are in the optional list and not strictly required for basic operation, but installing them is wise. For instance, `lubridate` could be used in date parsing if needed, and `data.table` can speed up data handling if you choose to use it for CSV filtering. They will also remove any warnings about "Package X not available" on startup.
- `shinyjs` – if any UI elements rely on it (the code loads it optionally [32], possibly for UI interactions). If not used, it's optional, but generally lightweight to include.
- `yaml` – optional (used if reading config files). Probably not critical unless you have a YAML config to parse.

How to install: Update the Docker build steps. In the `r-shiny-app/Dockerfile` (or equivalent), ensure the R packages installed cover all the above. The previous deployment used `.Rprofile` with `install.packages`. Double-check that script contains all needed names. If not, add lines for the missing ones. Alternatively, you can use a `renv.lock` with all dependencies pinned for reproducible environments. Since the image is based on `rocker/shiny:4.3.x`, CRAN installs should be smooth.

📦 **Version Mismatch (ggplot2/scales):** The code uses `ggplot2` for fallback charts and expects the pairing with `scales` for formatting. We did not see an explicit use of `scales::percent` or similar, but it's good to have `scales` installed to avoid any warnings (it's in optional list). Make sure the installed **ggplot2** version is recent (≥3.4.x) so that `geom_bar` and other functions behave as coded. If you notice any warnings like "`scales` not available for labeling" in logs or the charts lacking proper formatting, updating these packages will help.

📦 **Dependency Loading Order:** The global script loads **dplyr** before **data.table** (if data.table is loaded at all). Sometimes conflicts in function names (e.g., `data.table` and dplyr both have `between()`) can cause warnings. Currently, data.table is optional and likely not loaded, so no conflict arises. If you do load both, consider using `conflict_prefer()` or simply accept the messages. It's minor and doesn't break functionality.

Environment Variables: The **Railway environment** for the unified app should be reviewed:

- **Database credentials:** Confirm that either `DATABASE_URL` is set to the Postgres connection string or the `PGHOST, PGUSER, PGPASSWORD, PGDATABASE, PGPORT` are all set. In the provided screenshot, those appear to be present (perhaps from a Railway Postgres plugin or manually added). The app will log which path it took – e.g., *"Using DATABASE_URL for connection"* or *"Using individual PG* env variables"* on startup [17] [37]. Check the logs to ensure the pool was created successfully. If not, adjust as discussed (e.g., ensure `sslmode`). For Supabase: use the connection string with standard credentials (the audit docs suggested using the primary `postgres:...@host:5432/postgres` format [89] to avoid username parsing issues).

- **Demo mode:** Make sure `USE_DEMO_DATA` is **FALSE** in production (unless you explicitly want to run with the small demo dataset). If that was accidentally left true, the app would ignore the database and only use the tiny demo data, which is not desired for a live system. The UI's System Info panel will show "Data mode: Demo CSV" if that's the case [90]. For production, it should show "Data mode: Database".

- **Monitoring:** The `.env.example` suggests `ENABLE_MONITORING=true` [91]. You can keep this true – it doesn't harm anything as the app catches errors if the monitoring subsystem fails. With it on, you'll get log entries like *"Monitoring and logging system initialized"*. If you see any issues (like timeouts), you could turn it off, but given `start_monitoring()` is disabled, it's probably fine to leave on for now, to at least capture error logs via `log_error` calls (e.g., in `get_lexml_dashboard_metrics` when metrics calc fails [92]).

- **API Keys:** If the system is expected to pull live data (e.g., legislative updates from external APIs), ensure the API keys (Planalto, Câmara, Senado) are set in Railway env as per `.env.example` [93]. The R app might not yet use them (those might be legacy for the Python API), but if any module (like a LexML fetch or document viewer) needs them, having them configured is wise.

- **Misc:** Check that `ENVIRONMENT=production` is set (the Railway config in `railway.toml` might already do this). Also, the front-end integration: previously the React app used an environment flag to know if the R Shiny was up. In the unified setup, if the React front-end is deprecated, this is moot. But if it's still in use as a UI, verify it points to the new unified R service URL (which it likely does, per the resolution docs) and CORS is allowed for that domain (the example env had `CORS_ORIGIN=https://sofiadonario.github.io` [94] which should be fine).

Dockerfile & Build: The new unified app should have a Dockerfile similar to the legacy `Dockerfile.railway` [95] [96]. Key points to verify in it:
- All necessary files are copied into the image. Specifically, **include the** `data/lexml_unified_dataset.csv` (and any other CSVs) in the Docker build. If it's not in the image, the fallback will log "CSV file not found" [97] and your app will appear with no data when the DB is down. You might add `COPY data/ /app/data/` in the Dockerfile.
- The `.Rprofile` (or install script) is up-to-date with the package list discussed. If you're manually listing packages, ensure to add the missing ones. The Dockerfile snippet shows it sources `.Rprofile` after copying it [98]. If that profile was created earlier (perhaps by a script or manually), double-check its contents. It should contain `install.packages(c("shiny","shinydashboard","DT","plotly","dplyr","RColorBrewer","DBI","RPostgres" packages...))`. If not, update it and rebuild.
- Remove any leftover or unused pieces from the multi-service setup. For example, if the unified app no longer needs the Python service or Nginx, make sure the Dockerfile isn't still installing those or copying irrelevant files. The audit docs show the monorepo complexity was resolved, so likely you have a clean, single Dockerfile. Just something to be mindful of – a lean image with only R and required libraries will start faster and use less memory.

In conclusion, **most subsystems are working as intended** : The app successfully serves an interactive dashboard, swaps to fallback data on errors, and isolates each feature module. The **broken behaviors** ⚠ are primarily due to missing R packages or minor integration oversights, rather than logic flaws. By

installing the identified packages and tweaking a bit of server integration (for advanced search and status reporting), the production deployment should become fully reliable. The PostgreSQL connectivity is already robust (just ensure proper env config), and the caching/monitoring systems are in place, awaiting further tuning if needed. With these fixes: all modules will load (green checkmarks in the UI), the *Geographic Analysis* will display actual maps, the *Advanced Analytics* will show charts, and no "module not available" warnings should appear. The system will then truly reflect the " PRODUCTION READY" status noted in the internal docs [99], providing a seamless experience on the live Railway app.

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [58] [59] [60] [63] [64] [65] [66] [67] [68] [69] [70] [71] [74] [75] [76] [77] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [90] app.R
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/app.R

[16] [19] [27] [28] [29] [30] [31] [32] [33] [34] [35] [52] [57] [61] [62] [92] global.R
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/global.R

[17] [18] [20] [21] [22] [23] [24] [25] [26] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [53] [54] [55] [56] [72] [73] [97] data_service.R
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/modules/data_service.R

[78] [95] [96] [98] Dockerfile.railway
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/legacy/r-shiny/r-shiny-app/Dockerfile.railway

[89] ALTERNATIVE_DATABASE_URL.md
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/docs/ALTERNATIVE_DATABASE_URL.md

[91] [93] [94] .env.example
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/legacy/config/.env.example

[99] RAILWAY_RSHINY_DEPLOYMENT_RESOLUTION_REPORT.md
https://github.com/sofiadonario/monitor-legislativo-v4/blob/ff3e2e8b547b1bbc9a9394650fa26fc3d89d0dc1/docs/deployment/RAILWAY_RSHINY_DEPLOYMENT_RESOLUTION_REPORT.md